

5. (a) A class details what fields and methods an object will contain, and said method's implementation. An abstract class also contains fields, but contains methods with no implementation, and classes that inherit from it must implement these methods, it may also contain methods already implemented. An interface is very similar to an abstract class, except that they contain no state, and a class can implement multiple interfaces, but can only inherit from one abstract class.
- (b) Different subclasses of a parent class can override a method each with different implementations. When a method takes in an argument of type of the parent classes, and uses a method defined in the parent class, if an object of a subclass is passed it, when that method is called, it will use the subclass' implementation. Example provided in `DynamicPolymorphism.java`
- (c) This could potentially reduce the memory footprint of a subclass, as if it does not require all fields or methods of a parent class, then they don't need to be stored along with the subclass. However this could arguably defeat the point of inheritance, as if B is-a A, then a B likely requires all functionality of A. Also, if B selectively inherits from A, and doesn't inherit all of it's methods, if a B is passed into a method that uses that method of A, then the program will crash.
- (d) Create a class called `NSStudent` that inherits from `Student`, and overrides the `pass()` method, changing the logic to only require 10 ticks, then whenever `pass` is called, it will perform the respective logic for a student or NS student.
- (f) i. Get: $O(1)$
Add: $O(1)$ (amortized)
Delete: $O(N)$
Contains: $O(N)$
Insert: $O(N)$
- iv. While the time complexity of allocating new space on the heap cannot be easily determined at any given point, the overall complexity can still be analysed. If using the expand-by-one approach, when the every element in the array is used, every addition to the list will require a realloc in order to fit it inside the array, making addition $O(n)$, where n is the number of additions. If the size of the array is doubled each time, then addition becomes an $O(\log n)$ operation, as doubling the number of additions will only require one more realloc.
6. (a) Both methods (a) and (b) have the same cost in the marking process, as they just iterate over each object and check if it can still be reached, and marks it if it cannot. (a) has less of a performance cost when deleting than (b), as it does not compact the memory. Method (c) takes more of a performance hit for marking, as it has to perform a copy from one region to another, which is an expensive process, and update the references. However, (c) has the least performance cost associated with deleting, as it only needs to delete one region of memory.
- (b) Making immutable classes could have a negative impact on garbage collection, as whenever something needs to be changed about an immutable class, it cannot, so a copy needs to be made, and so more marking and deleting needs to be done by the GC. However, references inside an immutable class cannot change, so that avoids the problem of references needing to be changed when an object referenced in a class moves from one stage in the age hierarchy to another.
- (d) An empty interface that indicates to the programmer that the class should have that functionality.
- (e) iii. Inheritance can cause issues with copy constructors, as copy constructors are not inherited, so making a copy constructor for a subclass becomes cumbersome as all the code to copy from its superclass must be written for every subclass.



- iv. When an object is not a subclass and contains no reference types.
 - (f) Simply copying the `mData` field from one class to another will mean that the copied class still refers to the same array of integers, and so is still a shallow copy. To actually provide a deep `clone()` method, a new array has to be initialised in the copy, and every element of the array has to be manually copied over.
7. (a) A **Vector** is a wrapper around an array that grows when the vector's size exceeds that of the array it contains. It uses an integer index to access elements. Implements **Serializable**, **Cloneable**, **Iterable**, **Collection**, **List**, **RandomAccess**. It is thread-safe and implements a fail-fast iterator. The **ArrayList** class is roughly equivalent to **Vector** except for that fact that is unsynchronized and not thread safe. A **LinkedList** is a doubly-linked list that implements **List** as well as **Deque**, which **Vector** and **ArrayList** do not. Also implements a fail-fast iterator and is not thread safe. A **TreeSet**, unlike all the other data structures maintain an order of all the elements added to it, defined by natural ordering or a **Comparator**. Also, unlike the previous data structures, a **TreeSet** can only store unique values.
- (d) A **LinkedList** has $O(1)$ time to insert at the head and $O(1)$ time to remove from the back, as Java implements a doubly linked list, so both the head and tail nodes are stored for immediate access. A **ArrayList** will have an $O(n)$ insertion at the head, as all the existing values will need to be shifted by one unit (size of a reference) in memory. removal at the tail is also $O(1)$. Therefore, a **LinkedList** is the more performant choice.
 - (e) `s1 == s2` will return **false**, as the **new** keyword instructs the JVM to allocate new memory, and so `s1` and `s2` will exist at different addresses. Since strings are immutable in Java, when `String s3 = "Hi"` is ran, the string "Hi" will be created in the data segment, and any other references to it will point to the same memory address. Therefore, `s3 == s4` will return **true** as they refer to the same string in memory.
8. (a) The JVM translates uses of generics into explicit casts where needed. Primitives do not inherit from **Object**, and so are not a reference. A reference cannot be cast to an immediate value, only a reference can be cast to a different reference by telling the JVM to treat the memory differently. Instantiating objects of a template type is forbidden as the actual signature of the constructor of whatever type will be used will not be known, and so could easily cause the program to crash.
- (c) Wildcards represent an unknown type. However, unlike generics, repeated use of the wildcard does not have to represent the same type.
 - (d) This code fails to compile as the `addItem`s method uses the same generic type **T** for both arguments, when `doubles` and `numbers` are different types.
9. (b) The exception should be checked, as when a user tries to request an invalid transaction, the program should report to them that it is invalid, not just crash. Since unchecked exceptions are not caught, only checked ones are, a checked exception should be used.
- (d) The `powaux` method, instead of returning a value, throws an exception instead, which is caught by the calling method, and then the catch block can use the `getAns()` method to get the return value of `powaux`. This method does not deal with the case in which a negative value for `n` is passed in. This way of programming does not seem to provide any immediate benefit to just returning the actual value, there is a lot of extra boilerplate, and I cannot personally see a benefit.



- (e) An object can be created inside the scope of a `try...finally` block, and that block can wrap around all the code said object needs to be in existence for. When the object is no longer needed, stop writing code in the `try` block, and inside the `finally` block, run the code to delete all references associated with the object and free any resources it uses, so when the object is no longer needed, the "destructor" is immediately ran.
- (f) Trying to return inside a `try` block will cause an `UnreachableCodeError`, so the program jumps to the `finally` block, and executes what is in there.

